# Experiment:4

**Aim:** Practice Inheritance, Abstract Classes (**Vehicle Hierarchy**)

**Definition:**

- **Inheritance:** Mechanism where one class (child) inherits attributes and methods from another class (parent).
- **Abstract Class:** A class that cannot be instantiated directly and may contain abstract methods that must be implemented by subclasses.

This example demonstrates **inheritance, abstract classes, and method overriding**.

**Algorithm:**

1. Create an abstract class `Vehicle` with abstract method `fuel_type()` and a normal method `start()`.
2. Define child classes `Car` and `Bike` inheriting from `Vehicle`.
3. Implement the abstract method `fuel_type()` in both child classes.
4. Create objects of `Car` and `Bike`.
5. Call `start()` and `fuel_type()` methods on each object.

**Program:**

```
from abc import ABC, abstractmethod

# Abstract class

class Vehicle(ABC):

    def __init__(self, brand, model):

        self.brand = brand

        self.model = model

    def start(self):

        print(f"{self.brand} {self.model} is starting...")

    @abstractmethod

    def fuel_type(self):

        pass
```

Dept of AIML

```
# Car class inheriting Vehicle
class Car(Vehicle):
    def fuel_type(self):
        return "Petrol or Diesel"
# Bike class inheriting Vehicle
class Bike(Vehicle):
    def fuel_type(self):
        return "Petrol"
# Using the classes
car = Car("Toyota", "Corolla")
bike = Bike("Honda", "CBR500R")
car.start()
print("Car Fuel Type:", car.fuel_type())
bike.start()
print("Bike Fuel Type:", bike.fuel_type())
```

**OUTPUT:**

```
Toyota Corolla is starting...
Car Fuel Type: Petrol or Diesel
Honda CBR500R is starting...
Bike Fuel Type: Petrol
```

Dept of AIML